

DA6400: Reinforcement Learning

Programming Assignment 1

Prof. Balaraman Ravindran

Kalyan S ed23b051 , Amritam H ee23b004 , Chandran K na23b040

[Drive link](#)

1 Gridworld and Value Iteration

1.1 Introduction

Designing a MDP for solving a Gridworld for a drone tasked with firefighting, along with various obstacles and phases to fill water and travel from lake to fire and then again fire to lake. The basic constraints given are:

- 5X5 gridworld with lake at (0,0) and fire at (4,4)
- Special states at (1,2),(3,2) with smoke fumes and (2,4),(3,4) with boulders
- Rewards are -1 for each step, -10 for smoke state, -100 and terminate for boulders and +100 for fire state with water

1.2 Question 1

The State space in the MDP includes each state having:

- Drone Position: $(x, y) \in \{0, 1, 2, 3, 4\}^2$
- Water status:
 - 0 $\rightarrow$ No water
 - 1 $\rightarrow$ Water

Thus

$$S = \{(x, y, w) \mid x, y \in \{0, \dots, 4\}, w \in \{0, 1\}\}$$

The total number of states is

$$25 \times 2 = 50$$

Terminal states are:

- Any boulder cell $\rightarrow$ terminal (crash, -100)
- Fire zone (4,4) with water $\rightarrow$ terminal state $\{(4, 4, 1), (+100)\}$

The Action space A is defined as:

$$A = \{north, south, east, west, hover\}$$

Transition Probability model $P(s'|s, a)$ is defined in two models:

- Normal Cells:
 - 70% $\rightarrow$ intended direction
 - 10% $\rightarrow$ perpendicular direction 1
 - 10% $\rightarrow$ perpendicular direction 2
 - 10% $\rightarrow$ stay
- Smoke Cells:
 - 40% $\rightarrow$ intended direction
 - 10% $\rightarrow$ perpendicular direction 1
 - 10% $\rightarrow$ perpendicular direction 2
 - 40% $\rightarrow$ stay

Also, the hover state is deterministic:

$$P(s' = s \mid s, hover) = 1$$

Reward Function $R(s, a, s')$ is defined as:

- Non-terminal moves $\rightarrow$ -1
- Enter smoke $\rightarrow$ -10
- Enter Boulder $\rightarrow$ -100 (terminal)
- Reached (4, 4, 1) state $\rightarrow$ +100 (terminal)

Thus,

$$R = -1 - 10 \cdot 1_{smoke} - 100 \cdot 1_{boulder} + 100 \cdot 1_{goal}$$

Discount:

$$\gamma = 0.95$$

Transition and reward matrices for neighboring state from (3, 3).

States from (3, 3) state using action space A reachable in one step are:

$$\{(3, 2), (3, 4), (2, 3), (4, 3), (3, 3)\}$$

All diagonals have probability 0.

Action	(2,2)	(2,3)	(2,4)	(3,2)	(3,3)	(3,4)	(4,2)	(4,3)	(4,4)
N	0	0.1	0	0.7	0.1	0	0	0.1	0
S	0	0.1	0	0	0.1	0.7	0	0.1	0
E	0	0	0	0.1	0.1	0.1	0	0.7	0
W	0	0.7	0	0.1	0.1	0.1	0	0	0
H	0	0	0	0	1	0	0	0	0

Table 1: Transition probability table

Action	(2,2)	(2,3)	(2,4)	(3,2)	(3,3)	(3,4)	(4,2)	(4,3)	(4,4)
N	0	-11	0	-1	-1	-1	0	0	0
S	0	0	0	-1	-1	-1	0	-101	0
E	0	-11	0	-1	-1	0	0	-101	0
W	0	-11	0	0	-1	-1	0	-101	0
H	0	0	0	0	-1	0	0	0	0

Table 2: Reward sub-matrix R for the selected grid states

1.3 Question 3

(a) Resulting Optimal Value Function and Policy & Difference

For $\gamma = 0.3$:

- Most states have value ≈ -1.4
- Only states very close to the fire have positive value
- Policies appear less directed toward goals

This is fundamentally different from $\gamma = 0.95$, where:

- Values increase smoothly toward the lake and fire
- Policies clearly funnel toward the objectives

For a reward obtained after k steps,

$$\text{Discounted reward} = \gamma^k R$$

For the fire reward,

$$R = 100$$

With $\gamma = 0.3$:

k	Discounted reward
3	2.7
4	0.81
5	0.24

After 4–5 steps the reward becomes negligible while each step incurs a cost of -1 . Thus

$$V = -k + 100(0.3)^k$$

Example:

$$V = -6 + 100(0.3)^6 = -5.27$$

Thus moving far toward the goal is worse than staying near the current position.

Final Conclusion

- Large $\gamma \rightarrow$ far-sighted planning
- Small $\gamma \rightarrow$ short-sighted planning

(b) Behaviour Near Hazardous States

Hazard cells produce reward

$$R = -1 - 10 = -11$$

Bellman equation:

$$Q(s, a) = \sum_{s'} P(s'|s, a)[R + \gamma V(s')]$$

Since $\gamma = 0.3$, the immediate reward dominates.
Thus the agent strongly avoids smoke cells.

(c) Hovering at some state

$$Q_{hover}(s) = -1 + \gamma V(s)$$

Hovering occurs if

$$V(s) > V(s')$$

Since many states have similar values ≈ -1.4 , hovering may appear optimal.

(d) Other Interesting States

For far states:

$$V(s) = -1 + 0.3V(s)$$

$$V(s) = -1.43$$

Near fire states values become larger because

$$V(s) \approx 100(0.3) - 1$$

1.4 Question 4

(a) Difference Near Hazardous Cells

Entering smoke now gives

$$R = -1 - 90 = -91$$

Thus

$$Q_{smoke} = -91 + \gamma V$$

while normal move

$$Q_{normal} = -1 + \gamma V$$

Thus

$$Q_{normal} - Q_{smoke} = 90$$

Therefore the policy strongly avoids hazardous cells.

(b) Preference of Hovering

$$Q_{hover} = -1 + \gamma V$$

$$Q_{move} \approx -1 + \gamma V - 90p$$

If $p > 0$,

$$Q_{hover} > Q_{move}$$

Thus hovering may become optimal near hazards.

(c) Preference of Longer Path

Short path:

$$V_{short} \approx -91 + \gamma^k 100$$

Long safe path:

$$V_{long} \approx -(k + m) + \gamma^{k+m} 100$$

Since

$$91 \gg m$$

the longer safe path becomes preferable.

(d) Effect of Stronger Winds

When the wind becomes stronger, the transition probabilities change to 0.4 for the intended direction and 0.25 for each perpendicular direction. This increases the stochasticity of the environment and significantly raises the probability of drifting into hazardous cells. Since hazardous states incur a large penalty (-90), the expected value of actions near these cells decreases. As a result the optimal policy is more conservative.

2 TD-based Control in Acrobot

Environment and Discretisation

Acrobot-v1 is a two-link underactuated pendulum where torque is applied at the elbow joint to swing the tip above height $h = 1$. Episodes terminate when

$$-\cos \theta_1 - \cos(\theta_1 + \theta_2) > 1$$

or after 500 steps. Each step gives reward -1 ; the terminal step gives 0.

The observation vector is

$$\mathbf{o} = [\cos \theta_1, \sin \theta_1, \cos \theta_2, \sin \theta_2, \dot{\theta}_1, \dot{\theta}_2].$$

Each dimension is discretised into $n = 10$ bins using `numpy.linspace` and `numpy.digitize`, yielding

$$|\mathcal{S}| = 10^6, \quad \gamma = 0.99.$$

2.1 SARSA and Q-learning

Both algorithms store $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ in a table of size $(11^6 \times 3)$.

SARSA (on-policy)

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

Q-learning (off-policy)

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

SARSA evaluates the ε -greedy policy actually executed, producing conservative updates. Q-learning bootstraps from the greedy action and therefore learns more aggressively.

2.2 Return vs Episode Curves

2.2.1 Hyperparameter Tuning

We grid-search $\alpha \in \{0.1, 0.2, 0.3\}$ and $\varepsilon_0 \in \{0.01, 0.1, 1.0\}$ over 5000 episodes, ranking configurations by mean return of the final 100 episodes.

A fixed $\varepsilon > 0$ prevents convergence to the optimal greedy policy; We therefore apply linear decay

$$\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_{t-1} - \alpha_\varepsilon), \quad \varepsilon_{\min} = 0.01.$$

Table 3: Top configurations (mean return over last 100 episodes).

Algorithm	Rank	α	ε_0	Mean Return
SARSA	#1	0.2	0.01	-246.0
	#2	0.3	0.01	-253.0
	#3	0.1	0.01	-274.5
Q-learning	#1	0.2	0.01	-236.8
	#2	0.1	0.1	-253.3
	#3	0.1	0.01	-264.6

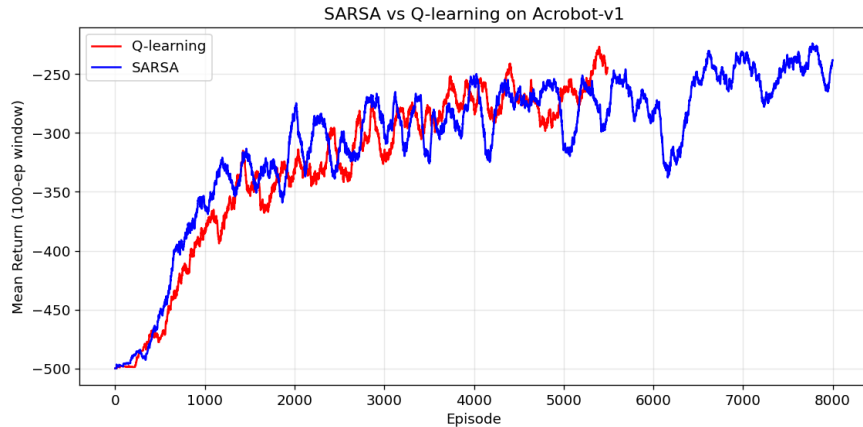


Figure 1: ε_0 decay from 1 to 0.01

2.2.2 Mean Performance over 10 Seeds

Initial Q-values are sampled from $\mathcal{U}(-0.01, 0.01)$ and environment states are randomised. The mean and 95% CI are

$$\bar{R}_t = \frac{1}{10} \sum_i R_t^{(i)}, \quad CI_t = 1.96 \frac{\hat{\sigma}_t}{\sqrt{10}}.$$

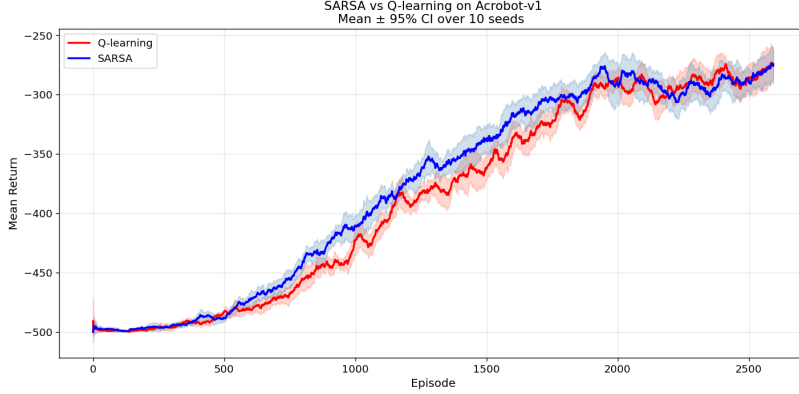


Figure 2: Mean return $\pm 95\%$ CI over 10 seeds.

Both algorithms improve steadily from an initial return near -500 , indicating fewer steps to reach the goal over time. The curves and confidence bounds are very similar.

The mean CI width for SARSA is 15.3 and for Q-learning 11.7, indicating slightly higher variance across SARSA runs. Both converge to similar asymptotic performance.

SARSA shows higher variance because its update uses the executed ε -greedy action $Q(s', a')$, so exploration noise affects targets. Q-learning uses $\max_{a'} Q(s', a')$, making updates less sensitive to stochastic actions.

2.3 ε Decay: $1 \rightarrow 0.1 \rightarrow \text{Fixed}$

ε starts at 1 and decays linearly by α_ε each episode until episode $T_{\text{decay}} = 1800$, after which it is fixed at 0.1.

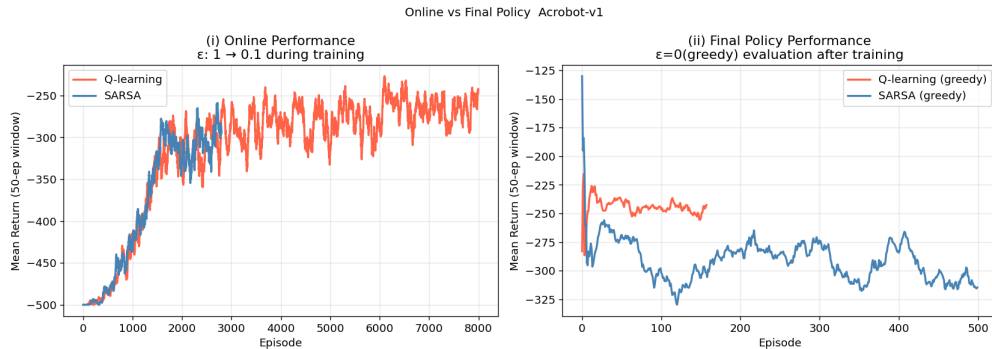


Figure 3: Online training (left) and greedy evaluation (right).

Both algorithms improve during training as ϵ decays from 1 to 0.1, since exploration gradually decreases and the agent exploits learned Q-values more often. However, online performance remains limited because the fixed 10% exploration still introduces random actions.

When evaluated greedily ($\epsilon = 0$), performance reflects the learned policy without exploration noise. Hence, Q-learning performs slightly better.

2.4 Effect of Number of Bins

State-space size grows rapidly with the number of bins: 5 bins $\rightarrow 1.6 \times 10^4$ states (very coarse), 10 bins $\rightarrow 1.0 \times 10^6$ states (balanced), 15 bins $\rightarrow 1.1 \times 10^7$ states (sparse visits), and 20 bins $\rightarrow 6.4 \times 10^7$ states (severe sparsity).

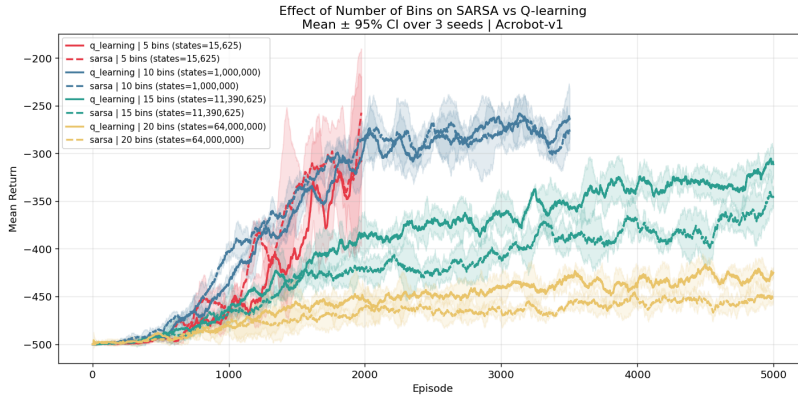


Figure 4: Q-learning performance across discretisations.

Coarse grids learn quickly but blur distinct states. Finer grids explode the state space, leaving most states unvisited within the episode budget. Empirically, Ten bins provide the best trade-off.

2.5 Modified Reward Function

Given reward

$$r = \frac{\eta h}{2} + \text{sign}(-1 + \eta h) \frac{2 - \eta h}{2}, \quad \eta > 0$$

which simplifies to

$$r = \begin{cases} \eta h - 1 & h < \frac{1}{\eta} \\ 1 & h \geq \frac{1}{\eta} \end{cases}$$

with saturation threshold $h^* = 1/\eta$.

2.5.1 Qualitative behaviour

Unlike the sparse reward $r = -1$, this reward increases with height h , giving a directional signal toward the goal and typically accelerating learning. However, if saturation occurs

before the goal, the agent may stabilise at intermediate heights instead of completing the task.

2.5.2 $\eta = 0.5$

$$r_{\eta=0.5}(h) = 0.5h - 1$$

Here $h^* = 2$ lies outside the reachable range, so no saturation occurs. The reward remains linear, providing a smooth gradient toward the goal and generally faster learning.

2.5.3 $\eta \in \{1, 2, 5\}$

$$r_{\eta=1}(h) = \begin{cases} h - 1 & h < 1 \\ 1 & h \geq 1 \end{cases}$$

Saturation occurs near the goal height.

$$r_{\eta=2}(h) = \begin{cases} 2h - 1 & h < 0.5 \\ 1 & h \geq 0.5 \end{cases}$$

Early saturation creates a suboptimal attractor.

$$r_{\eta=5}(h) = \begin{cases} 5h - 1 & h < 0.2 \\ 1 & h \geq 0.2 \end{cases}$$

Very early saturation weakens the learning signal.

Optimal: $\eta \approx 1$ is preferable because the reward saturates near the goal height ($h \approx 1$), encouraging the agent to stabilise around the target. Larger η values cause saturation at much smaller heights, creating attracting states where the agent may circle around.

2.5.4 Reward design insights

1. Dense rewards help learning when aligned with the objective.
2. Saturation before the goal can introduce competing attractors.
3. As shown by Policy invariance paper by Andrew Ng et al., only potential-based shaping

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

guarantees that the optimal policy remains unchanged.

2.6 Bonus: Empirical Support

We compare Eq. (1) with $\eta \in \{0.5, 1.0\}$ and a physics-informed reward:

$$r = 0.4h_{\text{tip}} + 0.02(a - 1)\dot{\theta}_1 + 0.01 \cos \theta_2 \max(0, -\cos \theta_1) - 0.04 \max(0, -\dot{\theta}_2)$$

where

$$h_{\text{tip}} = -\cos \theta_1 - \cos(\theta_1 + \theta_2).$$

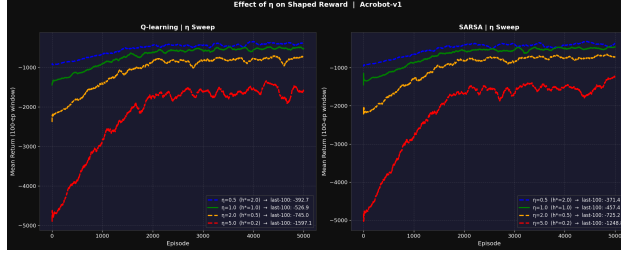


Figure 5: Learning curves under different reward functions.

Empirically $\eta = 0.5$ outperforms $\eta = 1$. Larger negative rewards early in training ($r \approx -3$ at $h = -2$) destabilise Q-updates, and the discontinuity at $h = 1$ is poorly represented under coarse discretisation. The linear $\eta = 0.5$ reward avoids both issues.

2.6.1 Physics-Based Reward

The reward combines task progress with dynamical cues:

$$r = \underbrace{0.4 h_{\text{tip}}}_{\text{height}} + \underbrace{0.02(a-1)\dot{\theta}_1}_{\text{resonance}} + \underbrace{0.01 \cos \theta_2 \cdot \max(0, -\cos \theta_1)}_{\text{alignment}} - \underbrace{0.04 \max(0, -\dot{\theta}_2)}_{\text{penalty}}$$

- $0.4h_{\text{tip}}$ encourages upward motion.
- $0.02(a-1)\dot{\theta}_1$ rewards torque applied in phase with link velocity.
- $0.01 \cos \theta_2 \max(0, -\cos \theta_1)$ favours link alignment near the neutral configuration.
- $-0.04 \max(0, -\dot{\theta}_2)$ penalises counterproductive motion of link 2.

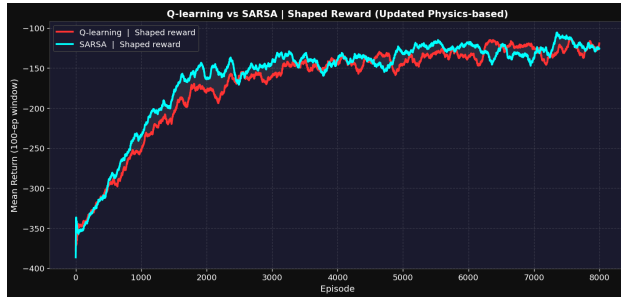


Figure 6: SARSA vs Q-learning with physics-informed reward.

2.6.2 Energy-Based Reward

The system state is defined by angles (θ_1, θ_2) and velocities $(\dot{\theta}_1, \dot{\theta}_2)$. The total mechanical energy E is the sum of kinetic energy T and potential energy V . The elements of the configuration-dependent inertia matrix $D(\theta)$ are:

$$\begin{aligned} d_{11} &= m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2 + 2l_1 l_{c2} \cos \theta_2) + I_1 + I_2 \\ d_{12} &= d_{21} = m_2(l_{c2}^2 + l_1 l_{c2} \cos \theta_2) + I_2 \\ d_{22} &= m_2 l_{c2}^2 + I_2 \end{aligned}$$

The total energy E and the target energy E_d (required to reach the upright equilibrium) are:

$$T = \frac{1}{2}(d_{11}\dot{\theta}_1^2 + 2d_{12}\dot{\theta}_1\dot{\theta}_2 + d_{22}\dot{\theta}_2^2) \quad (1)$$

$$V = -(m_1 l_{c1} + m_2 l_1)g \cos \theta_1 - m_2 l_{c2}g \cos(\theta_1 + \theta_2) \quad (2)$$

$$U_{upright} = (m_1 l_{c1} + m_2 l_1 + m_2 l_{c2})g \quad (3)$$

The final reward function R is formulated as:

$$R = -w_1|E - E_d| \quad (4)$$

Energy Error ($|E - E_d|$): This rewards the algorithm to be as close to the energy of the upright condition.

2.6.3 Return-Step Correlation

To verify alignment with the task objective, we compute the Pearson correlation between episode return and $-\text{steps}$.

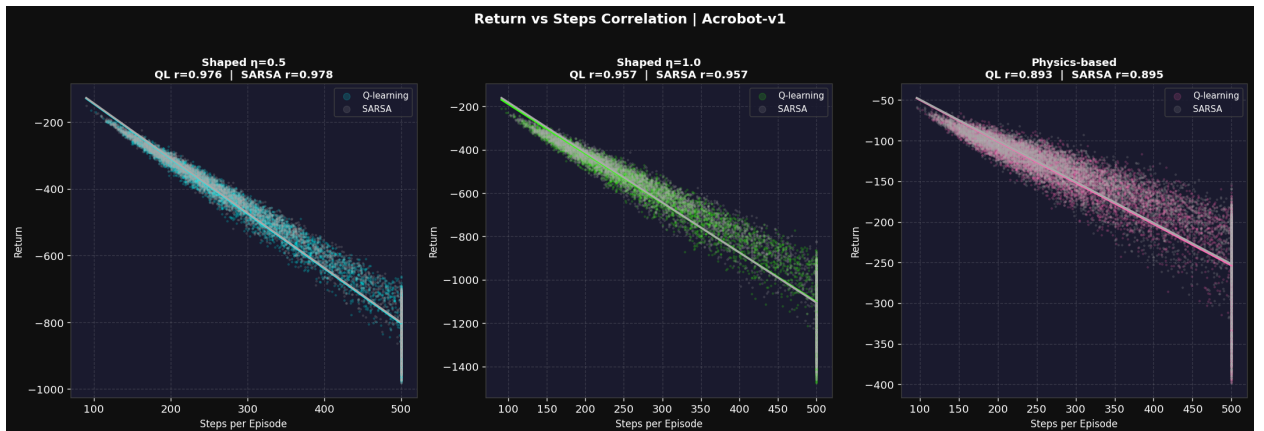


Figure 7: Return vs steps correlation.

All reward functions show strong correlation ($r > 0.8$), indicating that higher return consistently corresponds to fewer steps. $\eta = 0.5$ shows the strongest linear relationship because its return approximates a scaled step penalty.